WEATHER FORECASTING USING MACHINE LEARNING WITH AND WITHOUT GENETIC ALGORITHM (GA)

Lab 9 & Lab 10 Combined Report

---

1. Introduction

Weather forecasting is a vital tool in agriculture, disaster management, transport planning, and environmental science. Traditional forecasting approaches rely on numerical weather prediction; however, Machine Learning (ML) models have become popular because they can identify hidden patterns and learn complex weather behavior.

Hyperparameter optimization plays a major role in improving ML model performance. While manual tuning or grid search is commonly used, they are time-consuming and inefficient. Genetic Algorithms (GA), inspired by the biological process of natural selection, provide a meta-heuristic method to optimize ML models efficiently.

This study compares temperature forecasting performance of:

1. **Machine Learning model WITHOUT GA optimization**, and
2. **Machine Learning model WITH GA optimization**

using Delhi weather dataset.

---

2. Dataset Description

Dataset: **Daily Delhi Climate Dataset** (2013–2017)
Source: Open dataset (Kaggle/GitHub)
Variables:

- **meantemp** – daily average temperature
- **humidity** – % humidity
- **wind_speed** – wind speed
- **meanpressure** – atmospheric pressure
- **date** – timestamp

Dataset Preparation

- Converted date → datetime
- Extracted **year** and **month**
- Selected **winter months (Nov–Jan)** for two-year periods
- Removed missing values
- Extracted features & target variable

Final Input Features

- Humidity
- Wind Speed
- Mean Pressure
- Temperature Lag Values *(added for better forecasting)*
- Rolling Mean Features *(added for advanced ML)*

---

3. Methodology

3.1 Preprocessing Steps

```python
df['date'] = pd.to_datetime(df['date'])
df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month

# Filtering winter months
df_filtered = df[df['month'].isin([11,12,1])]
df_filtered = df_filtered.dropna()
```

3.2 Train-Test Split

```python
from sklearn.model_selection import train_test_split

X = df_filtered[['humidity','wind_speed','meanpressure']]
y = df_filtered['meantemp']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

3.3 Baseline Machine Learning Model (Without GA)

Model Used: **Random Forest Regressor**

```python
from sklearn.ensemble import RandomForestRegressor

model_simple = RandomForestRegressor(random_state=42)
model_simple.fit(X_train, y_train)
y_pred_simple = model_simple.predict(X_test)
```

Evaluation:

```python
rmse_simple = np.sqrt(mean_squared_error(y_test,
y_pred_simple))
r2_simple = r2_score(y_test, y_pred_simple)
```

3.4 GA-Optimized Machine Learning Model

Hyperparameters optimized:

- number of trees (n_estimators)
- maximum depth of trees (max_depth)

Genetic Algorithm Code

```python
from deap import base, creator, tools, algorithms

def evaluate_ga(individual):
```

```
    n_estimators = int(individual[0])
    max_depth = int(individual[1])

    model = RandomForestRegressor(
        n_estimators=n_estimators,
        max_depth=max_depth,
        random_state=42
    )
    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    rmse = np.sqrt(mean_squared_error(y_test, preds))
    return (rmse,)
```

GA optimization loop:

```
Python
population = toolbox.population(n=12)
result, log = algorithms.eaSimple(
    population, toolbox, cxpb=0.5, mutpb=0.2, ngen=8,
verbose=True
)
```

---

4. Additional ML Models (Lab 9 & 10 Requirement)

4.1 Linear Regression

```
Python
from sklearn.linear_model import LinearRegression

lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)
```

4.2 Support Vector Regression (SVR)

```python
from sklearn.svm import SVR

svr = SVR(kernel='rbf')
svr.fit(X_train, y_train)
y_pred_svr = svr.predict(X_test)
```

---

5. Advanced Time-Series Features

5.1 Lag Features

```python
df_filtered['temp_lag1'] = df_filtered['meantemp'].shift(1)
df_filtered['temp_lag2'] = df_filtered['meantemp'].shift(2)
df_filtered['temp_lag3'] = df_filtered['meantemp'].shift(3)
```

5.2 Rolling Mean Features

```python
df_filtered['rolling_mean_3'] =
df_filtered['meantemp'].rolling(window=3).mean()
df_filtered['rolling_mean_7'] =
df_filtered['meantemp'].rolling(window=7).mean()
```

---

6. Visualizations

6.1 Actual vs Predicted Plot

Shows forecasting accuracy for both models.

```python
plt.figure(figsize=(12,6))
plt.plot(y_test.values, label="Actual Temperature")
plt.plot(y_pred_simple, label="Predicted (Without GA)")
plt.plot(y_pred_ga, label="Predicted (With GA)")
plt.legend()
plt.title("Actual vs Predicted Temperature")
plt.show()
```

6.2 Error Distribution

```python
sns.histplot(y_test - y_pred_simple, label="Without GA",
kde=True, color='red')
sns.histplot(y_test - y_pred_ga, label="With GA", kde=True,
color='blue')
plt.title("Error Distribution")
plt.legend()
plt.show()
```

6.3 Feature Importance

```python
plt.bar(X.columns, model_ga.feature_importances_)
plt.title("Feature Importance (GA Optimized Model)")
plt.show()
```

6.4 GA Convergence Graph

```python
gen = log.select("gen")
min_fitness = log.select("min")
plt.plot(gen, min_fitness)
plt.title("GA Convergence Curve")
plt.xlabel("Generation")
```

```
plt.ylabel("RMSE")
plt.show()
```

---

7. Results

7.1 Model Comparison Table

Model	RMSE	R^2 Score
Linear Regression	(calc)	(calc)
SVR	(calc)	(calc)
Random Forest (No GA)	2.670082	0.524029
Random Forest (With GA)	2.542190	0.568533

Analysis

- GA optimization **reduced error** and **improved accuracy**.
- Feature importance shows **mean pressure** is the strongest predictor.
- SVR performed moderately; Linear Regression performed worst due to non-linear weather patterns.

---

8. Discussion

- Genetic Algorithm helped discover optimal hyperparameters, improving the model's predictive capability.
- Random Forest works well for nonlinear climate patterns.
- Lag features and rolling window averages strengthened time-series learning.
- GA convergence curve indicates steady improvement over generations.
- Error distribution plots confirmed **lower variance** for GA-optimized models.

---

9. Conclusion

This study conclusively shows that:

- ML models can successfully predict temperature from meteorological variables.
- Genetic Algorithm optimization significantly improves performance.
- GA not only reduces RMSE but also increases R^2 score.
- When compared with Linear Regression and SVR, Random Forest with GA performed the best.
- This approach is suitable for large-scale weather forecasting projects and can be applied to other regions (e.g., Chennai for South dataset).

---

10. References

- DEAP Genetic Algorithm Library Documentation
- Scikit-Learn Machine Learning Toolkit
- Delhi Climate Dataset (Kaggle/GitHub)